AP React – Partie 3
Requêtes API

Introduction

Nous allons maintenant faire des appels à une API pour récupérer les données et non plus les récupérer depuis un fichier statique. En effet, cela ne correspondait pas aux conditions réelles de fonctionnement d'une application.

Nous allons commencer par faire des requêtes API pour l'authentification, en récupérant le login et le mot de passe de la base de données. Puis, nous ferons des requêtes pour afficher les frais, les ajouter, les modifier et les supprimer (CRUD).

L'API est normalement accessible à l'adresse suivante : `'http://gsb.julliand.etu.lmdsio.com/api`

Vérifiez que l'API est accessible :

- Testez la route /api/visiteur/login avec Postman pour confirmer qu'elle renvoie bien un access_token et les données de l'utilisateur. Quel est le contenu de la réponse ? Notez la structure des données retournées.

Etape 1 : Installer Axios

Axios est une bibliothèque populaire pour effectuer des requêtes HTTP depuis un navigateur ou Node.js.

```
yarn add axios
```

Etape 2 : Créer un service d'API pour l'authentification

Créez un dossier services/ dans src/ et un fichier authService.js dans src/services/ :

1. Importez la bibliothèque Javascript axios
```
import axios from 'axios';
```
2. Définissez l'URL de notre API
```
const API_URL = 'http://gsb.julliand.etu.lmdsio.com/api/';
```
3. Définissez la fonction signIn, déclarée comme **asynchrone (avec async)**, ce qui signifie qu'elle peut contenir des opérations asynchrones (comme un appel à une API) et qu'elle renvoie une **promesse** (Promise). Le mot-clé await est utilisé pour **attendre** que la promesse renvoyée par axios.post() soit résolue. Pendant que la requête est en cours, le reste du code dans la fonction signIn est mis en pause jusqu'à ce que la réponse soit reçue.

   Si la réponse contient un access_token, cela signifie que l'authentification a réussi. Dans ce cas, on stocke les données de l'utilisateur dans le localStorage sous la forme de string.

La fonction signIn renvoie les données de la réponse (response.data) pour qu'elles puissent être utilisées par le code qui appelle cette fonction.

```
export const signIn = async (login, password) => {
  const response = await axios.post(`${API_URL}visiteur/login`, {login: login,
password: password });
  if (response.data.access_token) {
    localStorage.setItem('user', JSON.stringify(response.data.visiteur));
    localStorage.setItem('token', response.data.access_token);
  }
  return response.data;
};
```

Etape 3 : Modifier le contexte d'authentification (Context API)

Modifier src/context/AuthContext.jsx

1. Importer la fonction signIn du service :
```
import { signIn } from '../services/authService';
```
2. Déclarer un état token et l'initaliser avec la valeur null.
3. Remplacer la fonction loginUser par la fonction suivante :

```
const loginUser = async (login, password) => {
    const data = await signIn(login, password);
    setUser(data.visiteur);
    setToken(data.access_token);
    return data;
};
```

Cette fonction appelle la fonction signIn sur service (qui exécute l'appel API) puis modifie les états user et token avec les valeurs récupérées. Enfin, elle renvoie les données renvoyées par l'API.

4. Modifier l'objet value du return. Ici, les données et fonctions que l'on veut rendre disponibles aux composants enfants (notamment le composant Login) sont :
   - user : Les informations de l'utilisateur connecté (ex: { id_visiteur: 2, nom_visiteur: "Andre", ... }).
   - token : Le access_token utilisé pour authentifier les requêtes futures (ex: "4|NKf4ZhPHhoK9XwNz5MJKaBZwcGypk5g3QoBbSTBy8ff6fa36").
   - loginUser : la fonction pour connecter l'utilisateur.

```
return (
    <AuthContext.Provider value={{ user, token, loading, loginUser }}>
      {children}
    </AuthContext.Provider>
  );
```

Etape 4 : Modifier la fonction handleSubmit du composant Login

1. Déclarer la fonction handleSubmit du composant Login comme asynchrone car elle va appeler une fonction asynchrone (loginUser) :

```
const handleSubmit = async (e) => {
```

2. Remplacer le bloc if/else par un bloc try/catch.
   Le bloc try contient le code qui pourrait échouer :
   - l'appel à loginUser. Attention, loginUser est une fonction asynchrone qui envoie une requête à une API (avec Axios). Il faut donc ajouter await devant l'appel pour attendre que la promesse renvoyée par loginUser soit résolue (c'est-à-dire que la requête à l'API soit terminée).
   - La redirection vers /dashboard en cas de succès

   Le bloc catch contient le code à exécuter si la requête échoue (par exemple, identifiants incorrects), que la promesse est rejetée et qu'une erreur est donc levée :
   - on affiche le message 'Echec de la connexion' dans une pop-up

3. Tester la connexion, normalement cela devrait fonctionner à ce stade.

Voici un résumé du flux d'authentification que vous venez de mettre en place :

Frontend (React)

Clic sur "Submit" → Appel à `loginUser` → Requête POST (Axios) → API Backend

API Backend → Réponse (succès/erreur) → Axios → Mise à jour du contexte et `localStorage`

Redirection ou affichage d'erreur

Etape 5 : Gérer la déconnexion

1. Dans authService.js, ajouter la fonction logout qui supprime user et token du localStorage :

```
export const logout = () => {
  localStorage.removeItem('user');
  localStorage.removeItem('token');
};
```

2. Dans AuthContext, modifier la fonction logout pour qu'elle réinitialise aussi l'état token et qu'elle appelle la fonction logout() du service :

```
const logoutUser = () => {
    logout();
    setUser(null);
    setToken(null);
  };
```

3. Modifier le return de AuthContext pour exposer également logoutUser aux composants enfants.
4. Tester la déconnexion.
5. Dans authService.js, ajouter deux fonctions qui permettront de récupérer le user et le token depuis le localStorage pour pouvoir utiliser leur valeur.

```
export const getCurrentUser = () => {
  return JSON.parse(localStorage.getItem('user'));
};

export const getAuthToken = () => {
  return localStorage.getItem('token');
};
```

6. Dans AuthContext.js, ajouter un effet de bord pour mettre à jour les états user et token à l'authentification. Insérer le code suivant avant la définition de loginUser :

```
useEffect(() => {
    const user = getCurrentUser();
    const token = getAuthToken();
    if (user && token) {
      setUser(user);
      setToken(token);
    }
}, []);
```

Etape 6 : Protéger les routes privées avec le token et gérer le rechargement de la page

1. Modifier src/components/PrivateRoute.jsx pour utiliser le token au lieu de user pour protéger la route du dashboard. Le principe est exactement le même qu'avec user : si token est défini, on renvoie le composant enfant, sinon on redirige vers la page de connexion car cela signifie que l'utilisateur n'est pas connecté et n'est pas autorisé à voir le dashboard.

Si vous actualisez la page Tableau de bord, vous devriez vos apercevoir que vous êtes redirigé vers le formulaire de connexion. Cela est lié à la protection de notre route /dashboard par le composant PrivateRoute. Le problème est que lors du chargement de la page, le localStorage, qui contient la valeur du user et du token, met un certain temps à être chargé. Donc, au moins où le composant PrivateRoute est rendered (affiché), le token n'est pas défini et nous sommes redirigés vers /login. Nous allons donc ajouter un état de chargement dans notre composant AuthProvider : tant que le contexte sera en cours de chargement, aucune redirection ne sera faite, puis quand il aura été chargé, on redirigera vers /login ou /dashboard en fonction de la valeur de token.

2. Dans AuthContext.jsx, ajouter un état loading, initialisé à true (comme vous l'avez fait dans FraisTable).
3. Dans le useEffect(), si user et token sont définis, passer la valeur de loading à false. Cela signifie que le chargement du contexte est terminé.
4. Dans les props renvoyées dans l'affichage à la fin, ajouter loading :

```
return (
    <AuthContext.Provider value={{ user, token, loading, loginUser, logoutUser }}>
      {children}
    </AuthContext.Provider>
  );
```

5. Dans PrivateRoute, récupérer la valeur de loading en plus du token lorsque vous faites appel à useAuth().
6. Puis, si loading est vrai, renvoyer une simple div d'affichage texte :

```
<div>Chargement...</div>;
```

Cela va permettre d'éviter une redirection en attendant que le contexte soit chargé.

Etape 7 : Récupérer les frais depuis la base de données via un appel API

Notre connexion à l'API est fonctionnelle, nous allons donc maintenant récupérer les frais depuis la base de données via l'API au lieu d'un fichier JSON statique pour les afficher dans le dashboard. Nous ne récupérerons que les frais du visiteur connecté.

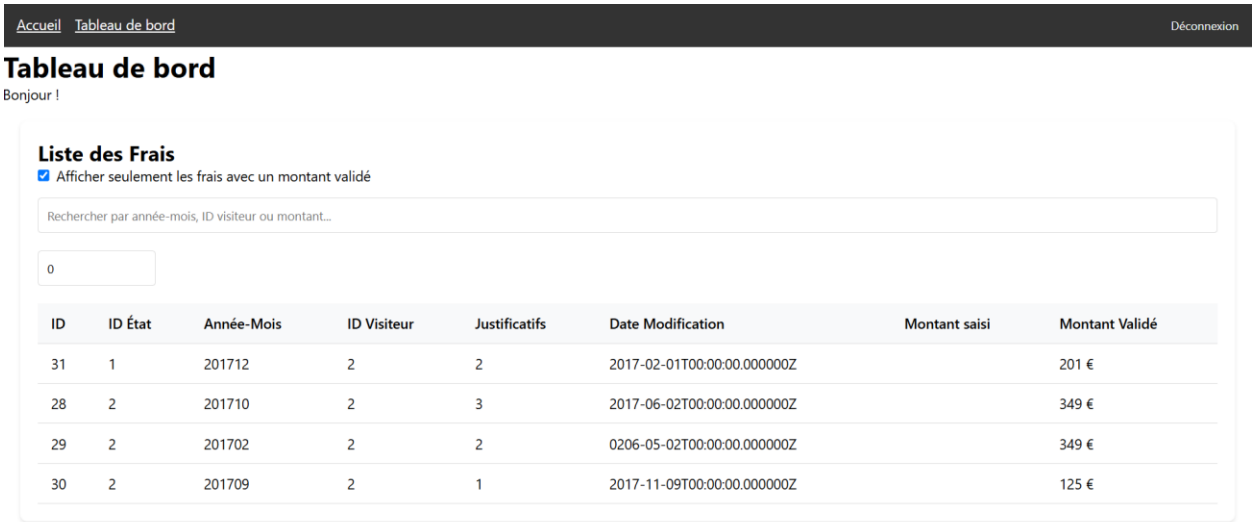
1. Importer axios dans FraisTable.jsx et supprimer l'import du fichier JSON.
2. Dans le composant FraisTable, nous aurons besoin de la valeur du user pour construire l'url de notre requête API.
   De plus, la requête API pour récupérer les frais est sécurisée, une vérification est faite sur le token de l'utilisateur connecté. Nous aurons donc besoin du token.
   Importer useAuth depuis AuthContext et récupérer user et token en faisant un appel à useAuth() dans le composant FraisTable.
3. Dans le useEffect, nous allons remplacer la simulation de chargement par un appel réel à l'API. Modifiez le useEffect en complétant le code suivant (en vous aidant du code que vous avez déjà) :
   Remarque : Pensez à importer API_URL depuis authService.js

```
  useEffect(() => {
    const fetchFrais = async () => {
      try {
        const response = await
axios.get(`${API_URL}frais/liste/${user.id_visiteur}`, {
          headers: {
            Authorization: `Bearer ${token}`,
          },
        });// Requête get à l'API à l'url
'http://gsb.julliand.etu.lmdsio.com/api/frais/liste/{id_visiteur}
```

```
      // TODO : Met à jour l'état avec les données de l'API
      // TODO : Met fin à l'état de chargement
    } catch (error) {
      console.error('Erreur lors de la récupération des frais:', error);
      // TODO : Arrête le chargement même en cas d'erreur
    }
  };

  fetchFrais(); // Appelle la fonction pour récupérer les données
}, []); // Tableau de dépendances vide = exécute une seule fois
```

4. Tester l'affichage des frais avec le visiteur de votre choix. Vous devriez obtenir l'affichage suivant avec le visiteur Andre :



Annexes :

Annexe 1 : Schéma du flux d'authentification

```
┌──────────────────────────────────────────────────────────────────────────────────────┐
│                          1. Clic sur le bouton "Submit"
│  - Le formulaire de connexion (Login.jsx) récupère les valeurs des champs
│    `login` et `password`.
│  - La fonction `handleSubmit` est appelée.
└──────────────────────────────────────────────────────────────────────────────────────┘
                            │
                            ▼
┌──────────────────────────────────────────────────────────────────────────────────────┐
│                          2. Appel à la fonction `loginUser`
│  - La fonction `loginUser` est appelée depuis le contexte `AuthContext`.
│  - Cette fonction utilise Axios pour envoyer une requête POST à l'API.
└──────────────────────────────────────────────────────────────────────────────────────┘
                            │
                            ▼
┌──────────────────────────────────────────────────────────────────────────────────────┐
│                          3. Envoi de la requête POST avec Axios
│  - Axios envoie une requête POST à `http://gsb.julliand.etu.lmdsio.com/api/visiteur/login`
│    avec les données : `{ login: "Andre", password: "secret" }`.
│  - Les en-têtes HTTP incluent `Content-Type: application/json`.
└──────────────────────────────────────────────────────────────────────────────────────┘
                            │
                            ▼
┌──────────────────────────────────────────────────────────────────────────────────────┐
│                          4. Traitement de la requête par l'API
│  - Le backend reçoit la requête et vérifie les identifiants.
│  - Si les identifiants sont valides, l'API génère un `access_token` et
│    renvoie une réponse JSON :
```

```
│   {
│     "visiteur": { "id_visiteur": 2, "nom_visiteur": "Andre", ... },
│     "access_token": "4|NKf4ZhPHhoK9XwNz5MJKaBZwcGypk5g3QoBbSTBy8ff6fa36",
│     "token_type": "bearer"
│   }
│  - Si les identifiants sont invalides, l'API renvoie une erreur (ex: 401).
└──────────────────────────────────────────────────────────────────────────────┘
                          │
                          ▼
┌──────────────────────────────────────────────────────────────────────────────┐
│                    5. Réception de la réponse par Axios
│  - Axios reçoit la réponse de l'API.
│  - Si la réponse est réussie (statut 200), Axios appelle le `.then()`.
│  - Si la réponse est une erreur (ex: statut 401), Axios appelle le `.catch()`.
└──────────────────────────────────────────────────────────────────────────────┘
                          │
                          ▼
┌──────────────────────────────────────────────────────────────────────────────┐
│                    6. Stockage des données dans le contexte
│  - Dans le `.then()`, la fonction `loginUser` du contexte est appelée pour
│    mettre à jour l'état global (`user` et `token`).
│  - Les données de l'utilisateur et le `access_token` sont stockés dans le
│    `localStorage` :
│    - `localStorage.setItem('user', JSON.stringify(response.data.visiteur))`
│    - `localStorage.setItem('token', response.data.access_token)`
└──────────────────────────────────────────────────────────────────────────────┘
                          │
                          ▼
┌──────────────────────────────────────────────────────────────────────────────┐
│                    7. Redirection ou affichage de succès
```

| - L'utilisateur est redirigé vers une page protégée (ex: `/`).

| - Un message de succès est affiché (ex: "Connexion réussie !").

└──┘